МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск подстроки в строке (КМП).

Студентка гр. 4345 _____ Жилина А. Е.

Преподаватель _____ Жангиров Т. Р.

Санкт-Петербург
2026

Задание

Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 25000$) и текста T ($|T| \leq 5000000$) найдите все вхождения P в T.

Вход:

- Первая строка — P

- Вторая строка — T

Выход:

индексы начал вхождений P в T, разделённые запятой; если P не входит в T, то вывести -1.

Test data

# Test	Input data	Output data
1	ab abab	0,2

Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$).

Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B, склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Вход:

- Первая строка — A

- Вторая строка — B

Выход:

Если A является циклическим сдвигом B, то индекс начала строки B в A; иначе вывести -1. Если возможно несколько сдвигов, вывести первый индекс.

Test data

# Test	Input data	Output data
1	defabc abcdef	3

Описание алгоритма

Основная идея алгоритма Кнута-Морриса-Пратта (КМП) заключается в использовании предварительно вычисленной информации о структуре самого искомого шаблона (pat) для оптимизации процесса поиска в тексте (txt). В отличие от наивных подходов, которые при каждом несовпадении совершают откат по тексту назад и сдвигают шаблон всего на одну позицию, алгоритм КМП исключает повторные проверки уже пройденных символов текста.

Вместо этого задача опирается на предварительный анализ шаблона. Алгоритм последовательно заполняет одномерный массив (вектор) префикс-функции pref, где каждая ячейка pref[i] хранит длину наибольшего собственного префикса подстроки шаблона (заканчивающейся в позиции i), который одновременно является и её суффиксом. Значение в каждой новой клетке массива префикс-функции вычисляется строго на основе уже известных предыдущих состояний.

Во время основного прохода по тексту, благодаря заранее вычисленному массиву pref, при возникникении несовпадения символов алгоритм точно знает, на какое максимальное расстояние можно безопасно сдвинуть шаблон вправо, не пропустив ни одного потенциального вхождения. Благодаря этому гарантируется, что указатель по тексту движется только вперед.

Вычисление префикс-функции: На этом этапе алгоритм проходит по шаблону, определяя длины совпадающих префиксов и суффиксов. Этот процесс опирается на метод двух указателей и работает за строго линейное время, так как внутренний цикл отката указателя в сумме совершает не более O(|P|) итераций за весь проход.

Поиск всех вхождений:

Для нахождения всех (в том числе перекрывающихся) вхождений шаблона в текст, алгоритм продолжает работу даже после полного совпадения. Когда длина текущего совпадения k достигает длины шаблона (patLen), индекс начала вхождения сохраняется в массив answers, а указатель k откатывается по префикс-функции (k = pref[k - 1]), что позволяет алгоритму продолжить поиск в оставшейся части текста без пропуска элементов.

Сложность по времени: O(|T| + |P|)

Основной цикл: Программа использует один независимый проход по искомому шаблону (строка pat) для построения префикс-функции, что занимает O(|P|) шагов, и один проход по исходному тексту (строка txt) для поиска, что требует O(|T|) итераций.

Вспомогательные операции: Обращения к массиву префикс-функции и сравнение символов происходят за константное время O(1). Операции логирования в файл добавляют константное время на каждую проверяемую позицию

Общая теоретическая сложность: O(|T| + |P|), так как внутренний цикл while в сумме по всем итерациям внешнего цикла for не может выполниться большее число раз, чем происходит сдвигов указателя вперед.

Сложность по памяти: O(|T| + |P|)

Основной объем памяти выделяется под сами исходные строки txt и pat, а также одномерный массив pref размером |P| для хранения префикс-функции шаблона. Стек и базовые переменные требуют O(1) памяти. Дополнительная память расходуется на вектор answers, в котором хранятся индексы всех найденных вхождений (в худшем случае, когда текст состоит только из искомых шаблонов, размер вектора может быть пропорционален длине текста). Итоговая сложность составляет O(|T| + |P|).

Для хранения промежуточных состояний алгоритм КМП не создаёт массив состояний, пропорциональный размеру входного текста. Вместо этого используется компактный вектор pref (префикс-функция), размер которого строго равен длине искомого шаблона pat. В этом массиве каждая ячейка pref[i] сохраняет уже найденную длину наибольшего собственного префикса, совпадающего с суффиксом для подстроки шаблона до индекса i включительно.

Этот массив выступает в роли «карты возвратов», позволяя алгоритму понимать, куда именно следует вернуть указатель длины совпадения k внутри шаблона при возникновении несовпадения в тексте. Такой способ хранения кардинально отличается от алгоритма Вагнера-Фишера (где требуется двумерная матрица), так как алгоритму КМП нужно помнить только внутреннюю структуру самого шаблона, а не историю преобразований всего текста.

Механизм работы префикс-функции и процесса поиска

Вместо заполнения громоздких структур состояний или использования бэктрекинга (возврата по тексту назад), алгоритм КМП в данной реализации опирается на итеративное вычисление длин совпадающих префиксов и суффиксов, хранящихся в векторе pref.

Процесс вычисления префикс-функции: Алгоритм последовательно сканирует искомый образец pat. Для построения функции используются два указателя: i (индекс текущего символа) и k (длина текущего максимального совпадающего префикса).

Если символы pat[i] и pat[k] совпадают, значение k увеличивается, и результат фиксируется в pref[i].

При возникновении несовпадения (и если k > 0) срабатывает цикл while, который откатывает указатель k к значению pref[k - 1]. Это позволяет алгоритму мгновенно «вспомнить» предыдущий подходящий префикс внутри текущей подстроки, не начиная проверку с нуля. Откат продолжается до тех пор, пока не будет найдено совпадение или пока k не станет равным 0.

Процесс поиска вхождений: Поиск по исходному тексту txt выполняется по аналогичной логике:

1. Указатель i перемещается по тексту txt строго слева направо.
2. Указатель k отслеживает текущую длину совпадения с шаблоном pat.
3. Если текущий символ текста txt[i] не совпадает с символом шаблона pat[k], указатель k совершает безопасный прыжок назад, используя данные из pref[k - 1].
4. Как только k достигает значения patLen (длины шаблона), это сигнализирует о нахождении полного вхождения. Программа вычисляет индекс начала вхождения (i - patLen + 1) и сохраняет его в вектор answers.
5. Чтобы найти все последующие (в том числе перекрывающиеся) вхождения, алгоритм не останавливается, а принудительно выполняет откат k = pref[k - 1]. Это позволяет продолжить поиск, рассматривая конец найденного вхождения как начало возможного нового совпадения.

Структуры данных и ключевые переменные:

pat (образец) и txt (текст) (string) — базовые структуры данных, представляющие искомую последовательность символов и строку, в которой осуществляется поиск.

pref (vector<int>) — основная структура данных алгоритма. Это одномерный динамический массив, размер которого строго равен длине шаблона pat. В нем кэшируются рассчитанные значения префикс-функции, что позволяет алгоритму мгновенно определять длину отката при несовпадении символов за константное время O(1).

k (int) — ключевая целочисленная переменная-указатель, отслеживающая текущую длину совпадения. Она выполняет двойную роль: на этапе построения массива pref переменная k определяет длину текущего совпадающего префикса-суффикса, а на этапе поиска в тексте указывает на индекс следующего ожидаемого символа в шаблоне.

answers (vector<int>) — динамический массив (вектор), накапливающий стартовые индексы всех найденных вхождений образца в текст. Использование этой структуры позволяет сохранить все позиции (включая перекрывающиеся) для их корректного вывода через запятую в конце работы программы.

log (ofstream) — служебный объект для работы с файловой системой, используемый для организации потоковой записи логов в файл log.txt. Это обеспечивает подробную трассировку работы алгоритма (итерации, откаты указателей, текущее состояние массива pref) без засорения стандартного потока вывода stdout.

Функции алгоритма:

ofstream log("log.txt")

Вспомогательный механизм для простой потоковой записи логов (аудита) в текстовый файл без засорения стандартного потока вывода stdout. Запись сообщений осуществляется напрямую через потоковый объект log.

vector<int> get_prefix_function(const string& pat) (блок вычисления внутри main)

Логический блок, реализующий предварительную обработку образца. Сканирует переданную строку pat и вычисляет динамический массив префикс-функции (pref), обращаясь к предыдущим состояниям за константное время.

void kmp_search(const string& pat, const string& txt)

Блок кода, реализующий основное сканирование текста с помощью алгоритма Кнута-Морриса-Пратта. Ищет все вхождения шаблона pat внутри текста txt, опираясь на массив pref, и сохраняет стартовые индексы всех совпадений в динамический массив answers.

Тестирование

Результаты тестирования представлены в табл. 1 и 2.

Таблица 1 – Результаты тестирования (программа 1, КМП)

№ п/п	Входные данные	Выходные данные	Комментарии
1.	aba abcaba	0,4	Верный результат
2.	aaa aaaaa	0,1,2	Верный результат
3.	xyz hello	-1	Верный результат
4.	saldfjksalkdfjsalfkdjsal kfdj xxx	-1	Верный результат

Таблица 2 – Результаты тестирования (программа 1, цикл. сдвиг)

№ п/п	Входные данные	Выходные данные	Комментарии
1.	defabc abcdef	3	Верный результат
2.	hello hello	0	Верный результат
3.	abcd dcba	-1	Верный результат
4.	abc abcd	-1	Верный результат

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: main.cpp

```
#include <iostream>
#include <string>
#include <vector>
#include <fstream>

using namespace std;

int main() {
    // Инициализация файла для записи логов
    ofstream log("log.txt");
    log << "--- Начало работы алгоритма КМП ---\n";

    // ускорение ввода-вывода
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // считываем образец и текст
    string pat, txt;
    getline(cin, pat);
    getline(cin, txt);

    log << "Считан образец (pat): '" << pat << "'\n";
    log << "Считан текст (txt): '" << txt << "'\n\n";

    int patLen = pat.size();
    int txtLen = txt.size();

    // построение префикс-функции для образца
    log << "--- Построение префикс-функции ---\n";
    vector<int> pref(patLen, 0);
    log << "Инициализация массива pref нулями. Длина массива: " << patLen
<< "\n";

    for (int i = 1; i < patLen; ++i) {
        log << "\nИтерация i=" << i << ", текущий символ pat[" << i <<
"] = '" << pat[i] << "'\n";
        int k = pref[i - 1];
        log << "  Начальное значение k (из pref[" << i - 1 << "]): " <<
k << "\n";

        while (k > 0 && pat[i] != pat[k]) {
            log << "  Несовпадение: pat[" << i << "] ('" << pat[i] <<
"') != pat[" << k << "] ('" << pat[k] << "')\n";
            log << "  Откат k к pref[" << k - 1 << "]\n";
            k = pref[k - 1];
            log << "  Новое значение k = " << k << "\n";
        }
        if (pat[i] == pat[k]) {
            log << "  Совпадение: pat[" << i << "] == pat[" << k << "]
('" << pat[i] << "'). Увеличиваем k.\n";
            ++k;
        } else {
            log << "  Совпадений нет, k остается равным " << k << "\n";
```

```
        }
        pref[i] = k;
        log << "  -> Записываем pref[" << i << "] = " << k << "\n";

        // Фиксация текущего вида массива префикс-функции
        log << "  Текущий массив pref: [";
        for (int j = 0; j < patLen; ++j) {
            log << pref[j] << (j == patLen - 1 ? "" : ", ");
        }
        log << "]\n";
    }

    // поиск всех вхождений образца в тексте (алгоритм КМП)
    log << "\n--- Поиск вхождений образца в тексте ---\n";
    vector<int> answers;
    int k = 0; // длина текущего совпадения
    log << "Начальная длина совпадения k = 0\n";

    for (int i = 0; i < txtLen; ++i) {
        log << "\nИтерация по тексту i=" << i << ", текущий символ txt["
<< i << "] = '" << txt[i] << "'\n";
        log << "  Текущая длина совпадения k = " << k << "\n";

        while (k > 0 && txt[i] != pat[k]) {
            log << "  Несовпадение: txt[" << i << "] ('" << txt[i] <<
"') != pat[" << k << "] ('" << pat[k] << "')\n";
            log << "  Откат k к pref[" << k - 1 << "]\n";
            k = pref[k - 1];
            log << "  Новое значение k = " << k << "\n";
        }
        if (txt[i] == pat[k]) {
            log << "  Совпадение: txt[" << i << "] == pat[" << k << "]
('" << txt[i] << "'). Увеличиваем k.\n";
            ++k;
        } else {
            log << "  Совпадений нет, k остается равным " << k << "\n";
        }
        if (k == patLen) {
            log << "  !!! Найдено полное вхождение образца (k == " <<
patLen << "). Индекс начала: " << (i - patLen + 1) << "\n";
            answers.push_back(i - patLen + 1);

            log << "  Откат k для поиска следующих вхождений к pref["
<< k - 1 << "]\n";
            k = pref[k - 1]; // продолжение поиска следующих вхождений
            log << "  Новое значение k = " << k << "\n";
        }
    }

    log << "\n--- Завершение алгоритма ---\n";
    log << "Всего найдено вхождений: " << answers.size() << "\n";
    log.close();

    // вывод результата
    if (answers.empty()) {
        cout << -1 << "\n";
    } else {
        for (size_t idx = 0; idx < answers.size(); ++idx) {
```

```
            if (idx > 0) cout << ",";
            cout << answers[idx];
        }
        cout << "\n";
    }

    return 0;
}
```

Название файла: main2.cpp

```
#include <iostream>
#include <string>
#include <vector>
#include <fstream>

using namespace std;

int main() {
    // Инициализация файла для записи логов
    ofstream log("log2.txt");
    log << "--- Проверка на циклический сдвиг (алгоритм КМП) ---\n";

    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string A, B;
    getline(cin, A);
    getline(cin, B);

    log << "Считана строка A: '" << A << "'\n";
    log << "Считана строка B: '" << B << "'\n\n";

    size_t n = A.size();
    size_t m = B.size();

    log << "Длина строки A (n) = " << n << "\n";
    log << "Длина строки B (m) = " << m << "\n\n";

    // циклический сдвиг возможен только при равной длине
    if (n != m) {
        log << "Длины строк не совпадают (n != m). Циклический сдвиг
невозможен. Возврат -1.\n";
        log.close();
        cout << -1 << "\n";
        return 0;
    }

    // обработка пустых строк (обе пусты - сдвиг 0)
    if (n == 0) {
        log << "Обе строки пусты (n == 0). Сдвиг равен 0. Возврат 0.\n";
        log.close();
        cout << 0 << "\n";
        return 0;
    }

    // для проверки на циклический сдвиг будем искать B в удвоенной
строке A
    string text = A + A;
```

```
    log << "Создана удвоенная строка A (text) для поиска: '" << text <<
"'\n\n";

    // построение префикс-функции для B
    log << "--- Построение префикс-функции для строки B ---\n";
    vector<size_t> pi(m, 0);
    log << "Инициализация массива pi нулями. Длина массива: " << m <<
"\n";

    for (size_t i = 1; i < m; ++i) {
        log << "\nИтерация i=" << i << ", текущий символ B[" << i << "]
= '" << B[i] << "'\n";
        size_t j = pi[i - 1];
        log << "  Начальное значение j (из pi[" << i - 1 << "]): " << j
<< "\n";

        while (j > 0 && B[i] != B[j]) {
            log << "  Несовпадение: B[" << i << "] ('" << B[i] << "') !=
B[" << j << "] ('" << B[j] << "')\n";
            log << "  Откат j к pi[" << j - 1 << "]\n";
            j = pi[j - 1];
            log << "  Новое значение j = " << j << "\n";
        }
        if (B[i] == B[j]) {
            log << "  Совпадение: B[" << i << "] == B[" << j << "] ('"
<< B[i] << "'). Увеличиваем j.\n";
            ++j;
        } else {
            log << "  Совпадений нет, j остается равным " << j << "\n";
        }
        pi[i] = j;
        log << "  -> Записываем pi[" << i << "] = " << j << "\n";

        // Фиксация текущего вида массива префикс-функции
        log << "  Текущий массив pi: [";
        for (size_t k = 0; k < m; ++k) {
            log << pi[k] << (k == m - 1 ? "" : ", ");
        }
        log << "]\n";
    }

    // поиск первого вхождения B в text (КМП)
    log << "\n--- Поиск первого вхождения B в text (КМП) ---\n";
    size_t j = 0;
    log << "Начальная длина совпадения j = 0\n";

    for (size_t i = 0; i < text.size(); ++i) {
        log << "\nИтерация по тексту i=" << i << ", текущий символ text["
<< i << "] = '" << text[i] << "'\n";
        log << "  Текущая длина совпадения j = " << j << "\n";

        while (j > 0 && text[i] != B[j]) {
            log << "  Несовпадение: text[" << i << "] ('" << text[i] <<
"') != B[" << j << "] ('" << B[j] << "')\n";
            log << "  Откат j к pi[" << j - 1 << "]\n";
            j = pi[j - 1];
            log << "  Новое значение j = " << j << "\n";
        }
```

```
        if (text[i] == B[j]) {
            log << "  Совпадение: text[" << i << "] == B[" << j << "]
('" << text[i] << "'). Увеличиваем j.\n";
            ++j;
        } else {
            log << "  Совпадений нет, j остается равным " << j << "\n";
        }
        if (j == m) {
            // Вхождение найдено, печатаем результат
            log << "  !!! Найдено полное вхождение строки B (j == " <<
m << "). Индекс начала (сдвиг): " << (i - m + 1) << "\n";
            log << "--- Завершение алгоритма (успех) ---\n";
            log.close();
            cout << i - m + 1 << "\n";
            return 0;
        }
    }
    // Вхождение не найдено
    log << "\nВхождение строки B в text не найдено.\n";
    log << "--- Завершение алгоритма (неудача) ---\n";
    log.close();
    cout << -1 << "\n";
    return 0;
}
```